
Use Cases

for

<WayFinder>

Version 1.0 approved

Prepared by <author>

<organization>

<27 August 2025>

1. Register New Account

Use Case ID:	REG01		
Use Case Name:	Register New Account		
Created By:	Das Srisha	Last Updated By:	Saba Azad
Date Created:	27 August 2025	Date Last Updated:	24 September 2025

Actor:	Initiating Actor: New User (first-time visitor) Supporting Actors: Clerk Authentication SDK for Next.js , Application Database
Description:	A first-time user registers for a new account using the Clerk SDK Sign-Up flow, which supports email/password registration as well as an OAuth-based identity provider (Google). Clerk handles credential validation, email verification, and session management.
Preconditions:	1. The user is connected to the internet. 2. The user does not already have a registered account with the same email or OAuth provider. 3. The Clerk SDK is properly initialized within the application.
Postconditions:	1. The user shall be successfully registered for an account. 2. The user's identity (email or OAuth ID) is securely stored in Clerk's managed user database. 3. A corresponding User object is created in the application's internal database 4. An authenticated session is created. 5. Upon successful registration, the user is redirected to the main map page or to the URL they were at before the sign-in flow (contained in search parameters).
Priority:	High
Frequency of Use:	Low
Flow of Events:	1. After the new user launches the app, the Login Page is displayed. 2. The user clicks the "Create Account" button on the Login Page to create an account. 3. The user will be redirected to another page to register for an account. User will be required to enter verifiable information, namely, email address and password or OAuth primary identifier (Google). 4. All fields will be validated on the frontend before enabling the Submit button. 5. If the user entered their email, email verification will be sent to the user containing a 6-digit verification code. If the operation was a success, the user will be successfully logged in. 6. The system redirects the user to the main map page or to the URL they were at before the sign-in flow (contained in search parameters).

Alternative Flows:	<u>AF-S2: The user tries to log in without a registered account</u> 1. If the user tries to log in without a registered account, an error message will be displayed stating that there is no such registered account. This message will also contain directions to a hyperlink that prompts the user to create an account. 2. The system returns to step 3. <u>AF-S4.1: The user inputs an email ID that is not valid</u> 1. The system displays the message “Please enter a valid email ID”. 2. The system returns to Step 3 and waits for inputs from the user. <u>AF-S4.2: The user inputs an email ID that is already associated with an existing account</u> 1. The system displays the message “Email ID is already associated with an account”. 2. The system returns to the Login page. <u>AF-S4.3: User signs up with multiple providers (Google + Email)</u> 1. The user first registers using a Google account (OAuth). 2. Later, the same user attempts to sign up with the email address linked to their Google account. 3. Clerk detects that the email is already associated with the existing Google identity. 4. Clerk automatically links both sign-up methods into a single user identity. 5. The registration is accepted, and the user can now log in using either Google or email/password. 6. The system redirects the user to Step 6 (successful login). <u>AF-S5: If the operation fails</u> 1. A toast message will be displayed indicating the corresponding server error. 2. The application returns to Step 3.
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error message “Network connection lost”. 2. The system returns to the Login Page. <u>EX 2: The application is unable to contact the database to register an account</u> 1. A server error toast is displayed. 2. The system will return to step 3.
Includes:	
Special Requirements:	1. Secure transmission of credentials over HTTPS. 2. Compliance with application’s authentication policies (e.g., password strength, supported OAuth providers).
Assumptions:	-
Notes and Issues:	-

2. Login

Use Case ID:	LOG01		
Use Case Name:	Login		
Created By:	Das Srisha	Last Updated By:	Saba Azad
Date Created:	27 August 2025	Date Last Updated:	24 September 2025

Actor:	Initiating Actor: New User (first-time visitor) Supporting Actors: Clerk Authentication SDK for Next.js
Description:	A user logs into their account using the Clerk SDK Sign-In flow, which supports email/password registration as well as an OAuth-based identity provider (Google). Clerk handles credential validation, email verification, and session management.
Preconditions:	1. The user must be connected to the internet. 2. The user must have an existing account in the system. 3. The user must not already be logged in to their account. 4. The Clerk SDK is properly initialized within the application.
Postconditions:	1. An authenticated session is created. 2. Upon successful authentication, the user is redirected to the main map page or to the URL they were at before the sign-in flow (contained in search parameters).
Priority:	High
Frequency of Use:	High
Flow of Events:	1. After the user launches the app, the Login Page is displayed. 2. User will be required to enter verifiable information, namely, email address and password or OAuth primary identifier (Google). 3. The system extracts the user's credentials from Clerk's managed user database. 4. If the user's credentials are valid and match an existing account, then the app logs the user in. 5. The system redirects the user to the main map page or to the URL they were at before the sign-in flow (contained in search parameters).
Alternative Flows:	<u>AF-S2.1: The user inputs an invalid email</u> 1. The system displays the message "Please enter a valid email ID". 2. Form submission does not occur. 3. The system returns to Step 2 and waits for inputs from the user. <u>AF-S2.2: The user inputs a wrong password or email</u> 1. The system displays an error message that says "Invalid credentials".

	- Successful form submission and redirection do not occur. - The system returns to Step 2 and waits for inputs from the user. <u>AF-S2.3: The user clicks on “Forgot Password”</u> - The system redirects the user to FGP01 - Forgot Password use case.
Exceptions:	<u>EX 1: If the system loses internet connection</u> The system displays the error toast that says “Network connection lost”.
Includes:	FGP01 - Forgot Password
Special Requirements:	- Secure transmission of credentials over HTTPS. - Compliance with application’s authentication policies (e.g., password strength, supported OAuth providers).
Assumptions:	-
Notes and Issues:	-

3. Forgot Password

Use Case ID:	FGP01		
Use Case Name:	Forgot Password		
Created By:	Das Srisha	Last Updated By:	Saba Azad
Date Created:	8 September 2025	Date Last Updated:	24 September 2025

Actor:	Initiating Actor: Registered User Supporting Actors: Clerk Authentication SDK for Next.js
Description:	A registered user who has forgotten their password can securely reset it using the application's password reset flow powered by Clerk SDK. The process involves entering the registered email, receiving a reset link, setting a new password, and regaining access to the system.
Preconditions:	1. The user must be connected to the internet. 2. The user must have an existing account registered with the system. 3. The Clerk SDK is properly initialized within the application.
Postconditions:	1. The user receives a reset link via their registered email. 2. The user resets their password. 3. The new password is securely updated in Clerk's managed user database. 4. The user can log in with their new password.
Priority:	High
Frequency of Use:	Low
Flow of Events:	1. The user clicks on the "Forgot Password" button on the Login Page. 2. The system (via Clerk) prompts the user to enter their registered email address. 3. The system (via Clerk) sends a reset password link to the entered email. 4. The user clicks the link and is redirected to a secure password reset page. 5. The user enters and confirms their new password. 6. Clerk validates the new password and updates the account. 7. The user is redirected to the Login Page with a success message: "Password reset successfully. Please log in with your new password".
Alternative Flows:	<u>AF-S3.1: The user does not click the reset link within the validity period</u> 1. The system displays the message "Reset link expired. Please request a new link." 2. The system returns to Step 1.
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error toast that says "Network connection lost".

	<u>EX 2: If the Clerk service is unavailable</u> 1. The system displays the message “Server error. Please try again later”.
Includes:	-
Special Requirements:	1. Secure transmission of credentials over HTTPS. 2. Compliance with application’s authentication policies (e.g., password strength, supported OAuth providers).
Assumptions:	-
Notes and Issues:	-

4. Logout

Use Case ID:	LOG02		
Use Case Name:	Logout		
Created By:	Das Srisha	Last Updated By:	Saba Azad
Date Created:	27 August 2025	Date Last Updated:	24 September 2025

Actor:	Initiating Actor: New User (first-time visitor) Supporting Actors: Clerk Authentication SDK for Next.js , Application Database
Description:	A logged-in user can securely log out of their account using the Logout action integrated via Clerk SDK. Logout is available from multiple UI entry points (responsive navbar and account page).
Preconditions:	1. The user must be connected to the internet. 2. The user must already be logged in to their account. 3. The Clerk SDK is properly initialized within the application.
Postconditions:	1. The user is logged out of their account. 2. Once logged out, the user session is terminated across the app. 3. The user is redirected to the Home page.
Priority:	High
Frequency of Use:	Low
Flow of Events:	1. The user clicks the “Log Out” button. 2. The system logs the user out of their account. 3. The system redirects the user to the Login Page.
Alternative Flows:	-
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error toast “Network connection lost”.
Includes:	-
Special Requirements:	1. Secure transmission of credentials over HTTPS. 2. Compliance with application’s authentication policies (e.g., password strength, supported OAuth providers).
Assumptions:	-
Notes and Issues:	-

5. Delete Account

Use Case ID:	DEL01		
Use Case Name:	Delete Account		
Created By:	Das Srisha	Last Updated By:	Saba Azad
Date Created:	27 August 2025	Date Last Updated:	24 September 2025

Actor:	Initiating Actor: New User (first-time visitor) Supporting Actors: Clerk Authentication SDK for Next.js , Application Database
Description:	A user can delete their account permanently.
Preconditions:	1. The user must be connected to the internet. 2. The user must already be logged in to their account.
Postconditions:	1. The user is logged out of their account. 2. The user is redirected to the Home page. 3. The data related to the user's account is deleted from the application database as well as Clerk's managed user database.
Priority:	High
Frequency of Use:	Low
Flow of Events:	1. The user clicks the "Delete Account" button in the Account page. 2. The system asks the user to confirm the deletion of their account. 3. The user clicks the "Confirm Delete Account" button. 4. The system logs the user out of their account. 5. The user is redirected to the Home page. 6. The data related to the user's account is deleted from the application database as well as Clerk's managed user database.
Alternative Flows:	<u>AF-S3: The user clicks the "Cancel" button instead</u> 1. The system stays on the same page and does not take any further action until the user triggers something else.
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error toast "Network connection lost".
Includes:	-
Special Requirements:	-

Assumptions:	-
Notes and Issues:	-

6. Live-location Access and receive Relevant Transport

Use Case ID:	LOC01		
Use Case Name:	Live-Location access and receive relevant transport		
Created By:	Das Srisha	Last Updated By:	Saba Azad
Date Created:	5 September 2025	Date Last Updated:	24 September 2025

Actor:	User (Initiating Actor), System, Browser Geolocation API, LTA API
Description:	The system requests permission to access the user's location and shows relevant nearby transport details.
Preconditions:	1. The user must be connected to the internet. 2. The user must have a valid account registered with the app. 3. The user must be logged into the application. 4. The user must be on the Home Page. 5. LTA API must be accessible for real-time data.
Postconditions:	1. The user will see location-based data around their current location.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. The user clicks on the Start Now button on the home screen and is redirected to the Map page. 2. The system requests location permission via the browser, and the user grants access. 3. The website zooms to the user's current location. 4. The website displays relevant nearby icons: Carpark icons, roadwork icons, taxi icons, and bus stop icons. 5. The user interacts with the Carpark icon, and the system will display the following information: Carpark name, number of available lots, type of lots (e.g. multi-storey/surface/sheltered/basement). 6. User clicks refresh, and the updated availability is fetched. 7. The user interacts with the Bus stop icon, and the system will display the following information: Bus stop name and code, next bus arrival timings, and bus diversions (if any). 8. The user clicks refresh, and the latest updated bus arrival times are fetched. 9. The user interacts with the Traffic alert icon, and the system will display the following information: Type of alert (lane closure/roadwork/accident), affected roads, and alert descriptions.

	10. User clicks refresh, and the latest alerts are fetched. 11. The user interacts with the Taxi icon just to show visual density.
Alternative Flows:	<u>AF-S2: The user denies access to live location</u> 1. The website displays the zoomed-out map of Singapore. 2. The user may manually zoom into a region. 3. The system displays the general traffic, carpark, taxi hotspots, and bus stop data for the zoomed-in area. 4. Clicking the carpark, bus stop, or traffic alert icons reveals the same detailed information as S5, S7, and S9, respectively.
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error toast “Network connection lost”. <u>EX 2: LTA API is unavailable</u> 1. The system displays the error toast “LTA API not available at the moment”. 2. The system returns to the Home Page. <u>EX 3: Browser blocks Geolocation</u> 1. The system defaults to the zoomed-out Singapore map.
Includes:	DATA01- Real-Time Transport Data
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

7. Real-Time Transport Data

Use Case ID:	DATA01		
Use Case Name:	Fetch and Update Real-Time Transport Data		
Created By:	Das Srisha	Last Updated By:	
Date Created:	5 September 2025	Date Last Updated:	

Actor:	User (Initiating Actor), System, LTA API
Description:	The system fetches real-time data from the LTA API to provide accurate and updated transport information to the user.
Preconditions:	1. The user must be connected to the internet. 2. The user must have a valid account registered with the app. 3. The user must be logged into the application. 4. The system must have access to the LTA API. 5. The user must be on the Maps Page or interacting with transport icons.
Postconditions:	1. The system updates and displays the latest transport data on the map. 2. The users can refresh data on demand.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. The user clicks on Start Now on the home screen and is redirected to the Maps page. 2. The user interacts with the transport icons, and the system triggers a call to the LTA API. 3. The API returns the latest transport data, including: bus arrival timings, taxi availability locations, carpark availability and traffic alerts. 4. The system updates the relevant icons on the map with new data. 5. The user clicks a transport icon, and the details are fetched from the API in real time and displayed. 6. The user clicks the Refresh button, and the system immediately requests updated information from the LTA API. 7. The data is refreshed automatically every minute, even if the user does not interact.
Alternative Flows:	<u>AF-S2: The user does not interact with any icons</u> 1. The system continues background updates every minute. <u>AF-S3: The data is temporarily missing for one category (e.g. Carparks only)</u> 1. The system updates all available data and displays “Latest Carpark data not

	available at the moment.” <u>AF-S6: The user refreshes before the 1-minute interval</u> 1. The system prioritises manual refresh and resets the countdown for the next update.
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error toast “Network connection lost” and pauses updates until reconnection. <u>EX 2: LTA API is unavailable</u> 1. The system displays the toast “LTA API not available at the moment. Displaying last known update”. <u>EX 3: API Response Timeout</u> 1. The system retries the request after 10seconds, up to 3 attempts.
Includes:	
Special Requirements:	1. Updates must not freeze or reload the entire page (use asynchronous API calls). 2. Automatic refresh should occur seamlessly in the background. 3. API response must be cached for fallback use if the API becomes temporarily unavailable.
Assumptions:	1. LTA API supports high-frequency requests without exceeding rate limits. 2. All transport data categories (bus, taxi, carpark, traffic) are available via API.
Notes and Issues:	1. Need to confirm API rate limits. 2. May need an error handling strategy for partial data updates.

8. Search Function

Use Case ID:	SRCH01		
Use Case Name:	Search Transport Information		
Created By:	Das Srisha	Last Updated By:	
Date Created:	5 September 2025	Date Last Updated:	

Actor:	User (Initiating Actor), System, LTA API
Description:	The user can search for transport-related data (carparks, bus stops, and traffic alerts) using keywords such as location, postal code, bus stop name/code, or road name.
Preconditions:	1. The user must be connected to the internet. 2. The user must be logged into the application. 3. The system must have access to the LTA API. 4. The user must be on the Maps Page to use the search bar on the Maps Page.
Postconditions:	1. The system retrieves results from the LTA API and displays the matching search results.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. The user clicks on the search bar. 2. To search for carparks, the user clicks on the carpark icon and enters either the location name or the postal code. 3. To search for bus stops, the user clicks on the bus stop icon and enters either the bus stop name or code. 4. To search for traffic alerts, the user clicks on the traffic alert icon and enters the road name. 5. The user clicks the Search button or presses Enter. 6. The system sends the query to the LTA API. 7. LTA API returns the matching data. 8. The system processes and displays the results. 9. The carpark results include the carpark name, address/postal code, number of available lots, and the type of lots. 10. The bus stop results include the bus stop name, bus stop code, next bus timings, and diversions (if any). 11. The traffic alert results include the type of alert and description.
Alternative Flows:	<u>AF-S2: The user enters a non-existent location name or postal code</u>

	1. The system displays the message “Please enter a valid location / postal code”. 2. The system returns to Step 2. <u>AF-S3: The user enters a non-existent bus stop name or code</u> 1. The system displays the message “Please enter a valid bus stop name/code”. 2. The system returns to Step 3. <u>AF-S4: The user enters a non-existent road name</u> 1. The system displays the message “Please enter a valid road name”. 2. The system returns to Step 4. <u>AF-S5.1: The user clicks the “Search” button without entering the location</u> 1. The system displays the message “Please enter a search term”. 2. The system returns to Step 1. <u>AF-S5.2: The user clicks the “Search” button without selecting an icon</u> 1. The system displays the message “Please select an icon to search”. 2. The system returns to Step 1.
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error toast “Network connection lost” and pauses updates until reconnection. <u>EX 2: LTA API is unavailable</u> 1. The system displays the toast “Search unavailable at the moment. Please try again later”. 2. The system returns to the Home Page. <u>EX 3: API Response Timeout</u> 1. The system retries up to 3 attempts. 2. The system displays the message “Search request timed out”. 3. The system returns to the Home Page.
Includes:	DATA01- Real-Time Transport Data
Special Requirements:	
Assumptions:	1. LTA API supports queries by postal code, bus stop code, and road name.
Notes and Issues:	1. Need to confirm whether traffic alerts by road name are supported in the LTA API. If not, the fallback may be filtering from all traffic alerts.

9. Travel-type Recommendation

Use Case ID:	REC01		
Use Case Name:	AI-Powered Travel Recommendations		
Created By:	Das Srisha	Last Updated By:	Saba Azad
Date Created:	5 September 2025	Date Last Updated:	24 September 2025

Actor:	User (Initiating Actor), System, LTA API, External Cost Estimation Service, AI model
Description:	The user enters an end location. The system processes real-time data (traffic, carpark availability, taxi density, bus timings, and cost estimates) through an AI model to recommend the optimal travel mode.
Preconditions:	1. The user must be connected to the internet. 2. The user must be logged in to their account. 3. The system must have access to the LTA API. 4. The system must have access to the AI model. 5. The system must have access to the Cost-Estimation service.
Postconditions:	1. The system displays a recommended travel type (Car, taxi, or bus). 2. The system provides reasoning (e.g., "Taxi recommended due to low congestion and high availability").
Priority:	High
Frequency of Use:	Moderate
Flow of Events:	1. The user navigates to the Travel Recommendation page. 2. The user enters an End location into the search bar. 3. The system retrieves the following real-time data via the LTA API: Road congestion levels, carpark availability near the destination, taxi availability near the origin, bus routes, arrival timings, and accessibility. 4. The system retrieves estimated travel times and costs for car, taxi, and bus. 5. The system passes all data into the AI model. 6. The AI model evaluates based on certain rules. 7. The AI model will recommend travelling by a Car if the roads are clear and carparks are available. 8. The AI model will recommend travelling by a Taxi if the availability is high and the roads are not congested. 9. The AI model will recommend travelling by a Bus → if a taxi is too expensive and the route is easily accessible by bus. 10. The AI model outputs the optimal travel mode along with supporting reasoning.

	11. The system displays the recommended mode and provides comparison info in the sidebar: estimated travel time, estimated cost and the reason for the recommendation.
Alternative Flows:	<u>AF-S2: The user enters an invalid or incomplete address</u> 1. The system displays the message “Please enter a valid destination”. 2. The system returns to Step 1. <u>AF-S3: The user clicks on the “Get Recommendation” button without entering the address</u> 1. The system displays the message “Please enter a valid destination”. 2. The system returns to Step 1. <u>AF-S4.1: Carparks near the destination are full</u> 1. The AI model rules out the “Car” option automatically. <u>AF-S4.2: No buses available on the route</u> 1. The AI model rules out the “Bus” option from recommendations. <u>AF-S4.3: No taxis available nearby</u> 1. The AI model rules out the “Taxi” option from recommendations.
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error toast “Network connection lost” and pauses updates until reconnection. <u>EX 2: LTA API is unavailable</u> 1. The system displays the error toast “Unable to fetch real-time data. Please try again later”. 2. The system returns to Step 1. <u>EX 2: Cost-estimation service is unavailable</u> 1. The system displays the message “Estimated fares not available, showing only travel time”. 2. The system returns to Step 1. <u>EX 3: AI model fails to return a recommendation</u> 1. The system defaults to show a comparison of all three modes without a recommendation..
Includes:	DATA01- Real-Time Transport Data, SRCH01- Search Transport Information, LOC01- Provide location access and receive relevant transport details
Special Requirements:	
Assumptions:	
Notes and Issues:	

10. View Profile

Use Case ID:	PRF01		
Use Case Name:	View Profile		
Created By:	Das Srisha	Last Updated By:	
Date Created:	5 September 2025	Date Last Updated:	

Actor:	User (Initiating Actor), Database
Description:	A user can view their personal information on their account page.
Preconditions:	1. The user must be connected to the internet. 2. The user must have a valid account registered with the app. 3. The user must already be logged in to their account.
Postconditions:	1. The system displays the user’s profile information.
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. The user clicks on the profile icon. 2. The system displays the profile information in the account page.
Alternative Flows:	-
Exceptions:	<u>EX 1: If the system loses internet connection</u> 1. The system displays the error toast “Network connection lost”.
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-